

## Sentinel v1.0 Technical Requirements Document (TRD)

### 1. Purpose

This document defines the technical architecture, infrastructure, services, data flow, security controls, operational requirements, and implementation requirements for Sentinel v1.0.

The TRD serves as the primary engineering reference document for development and deployment.

This document translates the Product Requirements Document (PRD) into a complete technical design.

---

### 2. System Overview

Sentinel is a Windows Endpoint Detection and Response (EDR) platform designed to provide:

- Endpoint Visibility
- Threat Detection
- Incident Correlation
- Threat Hunting
- Attack Graph Visualization

The platform consists of:

- Windows Endpoint Agent
- Event Normalization Layer
- Event Streaming Layer
- Detection Engine
- Threat Intelligence Engine
- Incident Correlation Engine
- Graph Engine
- Backend API
- Web Dashboard

Sentinel v1.0 supports Windows endpoints only.

---

### 3. Architecture Overview

#### High-Level Data Flow

Windows Endpoint



Sentinel Agent

↓

SUES Normalization Layer

↓

SQLite Buffer

↓

ASP.NET Core API

↓

Redis Streams

↓

Detection Pipeline

↓

PostgreSQL + Neo4j

↓

Dashboard API

↓

React Dashboard

---

## Primary Components

### Endpoint Layer

Responsible for:

- Telemetry Collection
- Event Normalization
- Offline Buffering
- Secure Transmission

---

### Processing Layer

Responsible for:

- Event Streaming
- Detection Execution
- Incident Correlation
- IOC Matching

---

## **Storage Layer**

Responsible for:

- Event Storage
- Alert Storage
- Incident Storage
- Relationship Storage

---

## **Presentation Layer**

Responsible for:

- Alert Monitoring
- Incident Investigation
- Threat Hunting
- Attack Graph Visualization

---

## **4. Architectural Principles**

### **Principle 1**

#### **Normalize Before Processing**

No component consumes raw telemetry.

All telemetry must first be transformed into the Sentinel Unified Event Schema (SUES).

---

### **Principle 2**

#### **Decouple Ingestion From Detection**

Event ingestion must remain independent from detection execution.

Redis Streams provide queue separation between ingestion and processing.

---

### **Principle 3**

#### **Store Relationships Separately**

PostgreSQL stores telemetry and incidents.

Neo4j stores entity relationships.

Raw telemetry shall never be stored in Neo4j.

---

## **Principle 4**

### **Security By Default**

All communications must use:

- TLS 1.3
- Certificate Pinning
- JWT Authentication
- Role-Based Access Control

---

## **Principle 5**

### **Graceful Degradation**

Failure of non-critical services shall not stop the platform.

Example:

If Neo4j becomes unavailable:

- Alerts continue working
- Incidents continue working
- Graph view becomes unavailable

---

## **5. Technology Stack**

### **Endpoint**

Language:

- C#

Framework:

- .NET 8

Application Type:

- Windows Service

Local Storage:

- SQLite

Telemetry Sources:

- ETW
- AMSI

- Sysmon
  - Windows Event Logs
- 

## Backend

Language:

- C#

Framework:

- ASP.NET Core

Logging:

- Serilog

API Style:

- REST
- 

## Frontend

Framework:

- React

Language:

- TypeScript

Styling:

- Tailwind CSS

Graph Visualization:

- Cytoscape.js
- 

## Data Layer

Relational Database:

- PostgreSQL

Graph Database:

- Neo4j Community Edition
- 

## Queue Layer

- Redis Streams

---

## Identity Layer

- Keycloak

---

## Deployment

- Docker
  - Docker Compose
- 

## 6. Endpoint Agent Architecture

### Responsibilities

The Sentinel Agent is responsible for:

- Telemetry Collection
  - Event Normalization
  - Offline Buffering
  - Secure Communication
  - Health Monitoring
- 

### Module Architecture

#### Telemetry Collector

Collects:

- ETW Events
  - AMSI Events
  - Sysmon Events
  - Windows Event Logs
- 

#### Normalization Engine

Converts raw telemetry into SUES events.

All downstream components consume only SUES events.

---

#### Local Buffer

Technology:

- SQLite

Stores:

- Pending Events
- Agent Metadata
- Agent Configuration
- Last Synchronization State

Purpose:

- Offline Operation
- Network Failure Recovery

---

## Communication Module

Responsibilities:

- HTTPS Communication
- Certificate Validation
- Event Upload
- Heartbeats
- Retry Logic

---

## Health Monitoring Module

Responsibilities:

- Service Health Monitoring
- API Connectivity Monitoring
- Telemetry Collection Monitoring
- SQLite Buffer Monitoring

Heartbeat Interval:

60 Seconds

Heartbeat Payload:

```
{  
  "endpointId": "endpoint-001",  
  "timestamp": "2026-01-01T10:00:00Z",  
  "status": "HEALTHY"
```

}

Supported States:

- HEALTHY
- DEGRADED
- OFFLINE

Definitions:

HEALTHY

All agent components operational.

DEGRADED

Partial functionality loss.

Examples:

- API unreachable
- Telemetry failure
- Buffer nearing capacity

OFFLINE

No heartbeat received.

---

## 7. Sentinel Unified Event Schema (SUES)

### Purpose

SUES serves as the canonical event model for Sentinel.

Every telemetry source must be transformed into SUES before entering the platform.

---

### Mandatory Fields

- eventId
- eventType
- source
- timestamp
- tenantId
- endpointId
- hostname
- userId

---

### Optional Fields

- processName
- processId
- parentProcessName
- parentProcessId
- commandLine
- domain
- ipAddress
- destinationPort
- protocol
- filePath
- registryKey
- severity
- alertId
- incidentId

Unknown fields shall be preserved but ignored by processing components.

---

### Supported Event Types

- PROCESS\_START
- PROCESS\_STOP
- NETWORK\_CONNECTION
- DNS\_QUERY
- FILE\_ACCESS
- FILE\_MODIFICATION
- REGISTRY\_MODIFICATION
- POWERSHELL\_EXECUTION
- ALERT
- INCIDENT

---

### Common Event Type Schemas

## PROCESS\_START

```
{  
  "eventType": "PROCESS_START",  
  "processName": "powershell.exe",  
  "processId": 1234,  
  "parentProcessName": "winword.exe",  
  "parentProcessId": 567,  
  "commandLine": "powershell -enc ..."  
}
```

---

## DNS\_QUERY

```
{  
  "eventType": "DNS_QUERY",  
  "domain": "evil.com",  
  "resolvedIp": "1.2.3.4"  
}
```

---

## NETWORK\_CONNECTION

```
{  
  "eventType": "NETWORK_CONNECTION",  
  "sourceIp": "10.0.0.5",  
  "destinationIp": "1.2.3.4",  
  "destinationPort": 443,  
  "protocol": "TCP"  
}
```

---

## FILE\_ACCESS

```
{  
  "eventType": "FILE_ACCESS",  
  "filePath": "C:\\Users\\User\\Downloads\\payload.exe",  
  "accessType": "READ"  
}
```

```
}
```

---

## REGISTRY\_MODIFICATION

```
{  
  "eventType": "REGISTRY_MODIFICATION",  
  "registryKey": "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run",  
  "operation": "CREATE"  
}
```

---

## ALERT

```
{  
  "eventType": "ALERT",  
  "alertId": "alert-001",  
  "ruleId": "DET-006",  
  "severity": "HIGH",  
  "mitreTechnique": "T1059.001"  
}
```

---

## 8. Event Processing Pipeline

### Step 1 – Collection

Telemetry is collected from:

- ETW
- AMSI
- Sysmon
- Windows Event Logs

---

### Step 2 – Normalization

Raw telemetry is converted into SUES format.

---

### Step 3 – Local Buffering

Events are written to SQLite.

Purpose:

- Offline operation
  - Reliability
- 

#### **Step 4 – API Ingestion**

Events are securely transmitted to the backend API.

---

#### **Step 5 – Redis Streams**

Events are published to Redis Streams.

---

#### **Step 6 – Detection Processing**

Detection Engine consumes events.

---

#### **Step 7 – Storage**

Results are stored in:

- PostgreSQL
  - Neo4j
- 

#### **Step 8 – Presentation**

Processed data becomes available through:

- Alerts
  - Incidents
  - Threat Hunting
  - Attack Graphs
  - Reports
- 

### **9. Detection Engine**

#### **Purpose**

The Detection Engine is responsible for identifying malicious or suspicious activity from normalized SUES events.

The engine consumes events from Redis Streams and generates alerts.

---

## **Responsibilities**

- Behavioral Detection
- IOC Matching
- MITRE ATT&CK Mapping
- Alert Generation
- Alert Deduplication

---

## **Detection Categories**

### **Initial Access**

- Office → PowerShell
- Office → CMD
- Office → WScript
- Office → MSHTA
- Suspicious PDF Reader Child Process

---

### **Execution**

- Encoded PowerShell
- PowerShell Download Cradle
- Rundll32 Abuse
- Regsvr32 Abuse
- Certutil Abuse
- MSHTA Abuse
- WMI Execution

---

### **Persistence**

- Run Registry Keys
  - Startup Folder Persistence
  - Scheduled Tasks
  - Service Creation
-

## **Defense Evasion**

- AMSI Tampering
  - PowerShell Obfuscation
  - Sysmon Disabled
  - Sentinel Service Disabled
- 

## **Credential Access**

- LSASS Access
  - Mimikatz Indicators
- 

## **Discovery**

- User Enumeration
  - Group Enumeration
  - Host Discovery
- 

## **Command & Control**

- DNS Tunneling Indicators
  - Beaconing Indicators
  - Known Malicious Domains
  - Known Malicious IPs
- 

## **Rule Execution Model**

Execution Strategy:

- Sequential
- Priority Ordered
- Multi-Match Enabled

Priority Order:

1. Critical
2. High
3. Medium
4. Low

A single event may trigger multiple detections.

Example:

Office → PowerShell

+

Encoded PowerShell

+

Malicious Domain

may generate three alerts.

---

### **Alert Deduplication**

Deduplication Window:

60 Seconds

Duplicate Criteria:

- Same Rule
- Same Endpoint
- Same Process

Result:

Duplicate alerts are collapsed into one alert.

Occurrence count is incremented.

---

### **Alert Generation SLA**

Measurement Boundary:

Normalized Event

↓

Redis Stream

↓

Detection Evaluation

↓

Alert Persisted

Target:

< 5 Seconds

Dashboard rendering is excluded.

---

### **Alert Output Schema**

Generated alerts contain:

- Alert ID
  - Rule ID
  - Severity
  - Endpoint
  - User
  - MITRE Technique
  - Timestamp
  - Evidence
- 

## **10. Sigma Engine**

### **Purpose**

Provide community-driven detection coverage using Sigma rules.

---

### **Architecture**

Sigma rules are converted into C# detection predicates during build time using a Sigma-to-Sentinel mapping tool.

Runtime Sigma compilation is not supported.

---

### **Responsibilities**

- Sigma Rule Execution
  - Sigma Alert Generation
  - Sigma Rule Validation
- 

### **Processing Flow**

Sigma Rule



Build-Time Conversion



C# Predicate



Detection Engine

---

#### Rule Sources

- Internal Rule Repository
  - Community Sigma Rules
- 

#### Dynamic Updates

Deferred to v1.1

---

### 11. Threat Intelligence Engine

#### Purpose

Provide IOC-based detection capabilities.

---

#### Supported IOC Types

- IP
  - DOMAIN
  - URL
  - HASH
- 

#### Upload Methods

##### JSON Upload

```
[  
  {  
    "iocType": "DOMAIN",  
    "value": "evil.com",  
    "severity": "HIGH"  
  }  
]
```

---

### CSV Upload

iocType,value,severity

DOMAIN,evil.com,HIGH

IP,1.2.3.4,CRITICAL

---

### Validation Rules

- Reject malformed entries
  - Reject unsupported IOC types
  - Deduplicate identical IOCs
- 

### Matching Strategy

Incoming telemetry is compared against IOC records.

Match Result:

IOC Match



Alert Generated

---

### Future Enhancements

Deferred:

- MISP Integration
  - STIX/TAXII
  - Abuse.ch
  - AlienVault OTX
- 

## 12. Incident Correlation Engine

### Purpose

Group related alerts into incidents.

---

### Correlation Criteria

An alert may join an existing incident when:

- Endpoint matches
  - User matches
  - Detection category matches
  - Time window matches
- 

### **Correlation Window**

Default:

15 Minutes

---

### **Edge Case Handling**

Alerts outside the window may still join an incident if:

- Endpoint matches
  - User matches
  - Attack chain indicators match
- 

### **Manual Operations**

Analysts may:

- Merge Incidents
  - Split Incidents
  - Reassign Alerts
- 

### **Incident States**

- Open
  - Investigating
  - Resolved
  - Closed
- 

### **Incident Severities**

- Low
- Medium
- High

- Critical
- 

### 13. PostgreSQL Architecture

#### Purpose

Primary operational datastore.

---

#### Stored Data

##### Users

- User Accounts
  - Roles
  - Authentication Metadata
- 

#### Endpoints

- Endpoint Inventory
  - Agent Versions
  - Status
- 

#### Events

- Normalized Events
  - Event Metadata
- 

#### Alerts

- Detection Results
  - Alert Metadata
- 

#### Incidents

- Correlated Alerts
  - Investigation Data
- 

#### Policies

- Agent Policies

- Configuration Profiles
- 

### **Threat Intelligence**

- IOC Records
- 

### **Audit Logs**

- Authentication Events
  - Administrative Actions
- 

### **Requirements**

- Indexed Queries
  - Foreign Keys
  - Backup Support
  - Tenant Isolation
- 

## **14. Neo4j Architecture**

### **Purpose**

Store entity relationships and attack paths.

Neo4j shall never store raw telemetry.

---

### **Node Types**

- User
  - Endpoint
  - Process
  - Domain
  - IP
  - File
  - Alert
  - Incident
- 

### **Relationship Types**

- LOGGED\_INTO
  - EXECUTED
  - CONNECTED\_TO
  - ACCESSED
  - GENERATED
  - PART\_OF
  - INVOLVES
- 

### **Graph Generation Workflow**

Alert Generated



PostgreSQL Persist



Relationship Extraction



Neo4j Update



API Response

Graph updates are synchronous in v1.0.

---

### **Failure Handling**

If Neo4j becomes unavailable:

- Alert Processing Continues
  - Incident Processing Continues
  - Graph Features Become Unavailable
- 

## **15. Redis Architecture**

### **Purpose**

Decouple ingestion from detection processing.

---

### **Technology**

## Redis Streams

---

### Stream Configuration

Stream Name:

sentinel-events

Consumer Group:

sentinel-detection-group

Consumers:

1 Detection Worker

---

### Acknowledgement Strategy

Manual Acknowledgement

---

### Message Retention

24 Hours

---

### Stream Trimming

Automatic trimming after retention period.

---

### Failure Handling

Unacknowledged messages remain pending.

Pending messages are reprocessed when workers recover.

---

### Future Enhancements

Deferred:

- Multiple Consumers
  - Distributed Detection Workers
  - Multiple Consumer Groups
- 

## 16. Authentication & Identity Architecture

### Identity Provider

## Keycloak

---

### Authentication

JWT-Based Authentication

---

### Authorization

Role-Based Access Control (RBAC)

---

### Supported Roles

#### Admin

Permissions:

- Full System Access
  - User Management
  - Policy Management
  - IOC Management
- 

#### Analyst

Permissions:

- View Alerts
  - Investigate Incidents
  - Threat Hunting
  - Reporting
- 

#### Viewer

Permissions:

- Read-Only Access
- 

### Keycloak Bootstrap

Automatic provisioning creates:

- Sentinel Realm
- Dashboard Client

- API Client
- Admin Role
- Analyst Role
- Viewer Role

---

### **Bootstrap Administrator**

Created automatically during first deployment.

---

### **Realm Import**

Realm configuration is imported automatically through Docker startup scripts.

No manual Keycloak configuration is required.

---

## **17. Multi-Tenant Architecture**

### **Purpose**

Support multiple organizations while maintaining strict tenant isolation.

---

### **Tenant Model**

Every organization is represented as a tenant.

Examples:

Tenant A

├— Users

├— Endpoints

├— Alerts

└— Incidents

Tenant B

├— Users

├— Endpoints

├— Alerts

└— Incidents

---

## **Tenant Identifier**

Every business object must contain:

tenantId

Examples:

- User
  - Endpoint
  - Event
  - Alert
  - Incident
  - Policy
  - IOC
- 

## **Isolation Enforcement**

Isolation is enforced at:

### **API Layer**

Every request must be scoped to the authenticated tenant.

---

### **Database Layer**

Queries must include tenant filtering.

Example:

WHERE tenant\_id = ?

---

### **Search Layer**

Threat hunting results are restricted to the requesting tenant.

---

## **Cross-Tenant Access**

Cross-tenant access is prohibited.

Exception:

Platform administrators with explicit system-level privileges.

---

## **18. Dashboard Architecture**

## **Purpose**

Provide analysts with visibility, investigation, and reporting capabilities.

---

## **Pages**

### **Overview**

Displays:

- Active Endpoints
  - Active Alerts
  - Open Incidents
  - Critical Incidents
  - Detection Trends
- 

### **Endpoints**

Displays:

- Endpoint Inventory
  - Health Status
  - Agent Version
  - Last Seen
- 

### **Alerts**

Displays:

- Active Alerts
  - Severity
  - Detection Rule
  - Endpoint
  - User
- 

### **Incidents**

Displays:

- Incident Summary
- Timeline

- Evidence
  - Notes
  - Related Alerts
- 

## **Threat Hunting**

Supports searches across:

- Users
  - Endpoints
  - Processes
  - Domains
  - IP Addresses
  - Alerts
  - Incidents
- 

## **Attack Graph**

Displays:

- Users
- Endpoints
- Processes
- Domains
- IPs
- Alerts
- Incidents

Visualization Technology:

- Cytoscape.js
- 

## **Reports**

Provides:

- Executive Reports
- Incident Reports
- Threat Reports

---

## Settings

Provides:

- Policy Management
- User Management
- IOC Management

---

## Dashboard Data Flow

React Dashboard



HTTPS



ASP.NET Core API



PostgreSQL / Neo4j

---

## Polling Strategy

Alerts:

5 Seconds

Incidents:

30 Seconds

Endpoints:

60 Seconds

Threat Hunting:

On Demand

Reports:

On Demand

Realtime WebSockets and Server-Sent Events are deferred to v1.1.

---

## 19. Security Architecture

### Purpose

Protect Sentinel infrastructure, data, and communications.

---

### **Transport Security**

All communications use:

- TLS 1.3
- 

### **Certificate Pinning**

Agent communication requires:

- TLS Validation
- Fingerprint Validation

Both checks must succeed.

---

### **Authentication**

Technology:

- JWT

Provider:

- Keycloak
- 

### **Authorization**

Model:

RBAC

Roles:

- Admin
  - Analyst
  - Viewer
- 

### **Password Security**

Algorithm:

Argon2id

---

### **Audit Logging**

Events logged:

- Login
  - Logout
  - Failed Authentication
  - User Creation
  - User Modification
  - Policy Changes
  - IOC Uploads
  - Administrative Actions
- 

### **Certificate Trust Model**

Agent installer contains:

- Trusted Server Identity
- Server Fingerprint

Connection Requirements:

1. TLS Validation Success
2. Fingerprint Validation Success

Both validations are mandatory.

---

### **Certificate Lifecycle**

v1.0:

- Static Deployment Certificate
- Manual Replacement

v1.1:

- Certificate Rotation
  - Automated Trust Updates
- 

## **20. Backup & Recovery**

### **Purpose**

Protect operational and investigative data.

---

## **PostgreSQL Backup**

Technology:

pg\_dump

Frequency:

Daily

Retention:

30 Days

---

## **Neo4j Backup**

Technology:

neo4j-admin database dump

Frequency:

Daily

Retention:

30 Days

---

## **Backup Storage**

Storage Location:

Docker Mounted Backup Volume

---

## **Recovery Objective**

Target Recovery Time:

24 Hours

---

## **Recovery Scope**

Recoverable Data:

- Users
- Endpoints
- Events
- Alerts
- Incidents

- Policies
  - IOCs
  - Graph Relationships
- 

## **21. Performance Requirements**

### **Endpoint Agent**

CPU Usage:

< 3%

RAM Usage:

< 200 MB

Startup Time:

< 10 Seconds

---

### **Platform**

Alert Generation:

< 5 Seconds

Dashboard Load:

< 3 Seconds

---

### **Scalability**

Validated Endpoint Count:

100+ Concurrent Endpoints

---

### **Database Performance**

Threat Hunting Queries:

< 3 Seconds

Incident Loading:

< 3 Seconds

Attack Graph Loading:

< 5 Seconds

---

## 22. Deployment Topology

### Containerized Services

Docker Compose deploys:

#### **sentinel-api**

Responsibilities:

- Event Ingestion
  - Detection Processing
  - Dashboard APIs
- 

#### **postgres**

Responsibilities:

- Operational Data Storage
- 

#### **neo4j**

Responsibilities:

- Relationship Storage
- 

#### **redis**

Responsibilities:

- Event Streaming
- 

#### **keycloak**

Responsibilities:

- Authentication
  - Authorization
- 

#### **dashboard**

Responsibilities:

- User Interface
- 

#### **Endpoint Agent**

Runs independently as:

Windows Service

Not containerized.

---

## **23. Failure Handling**

### **Agent Offline**

Behavior:

- Events buffered in SQLite
  - Upload resumes after connectivity returns
- 

### **API Failure**

Behavior:

- Agent retries uploads
  - Events remain buffered
- 

### **Redis Failure**

Behavior:

- Events retried until queue recovers
- 

### **PostgreSQL Failure**

Behavior:

- Events remain queued
  - Processing resumes after recovery
- 

### **Neo4j Failure**

Behavior:

- Alert Processing Continues
  - Incident Processing Continues
  - Graph Features Disabled
- 

### **Keycloak Failure**

Behavior:

- Existing sessions remain valid
  - New authentication requests fail
- 

## **24. Upgrade Strategy**

### **v1.0**

Upgrade Method:

Manual MSI Deployment

Administrators deploy newer agent versions manually.

---

### **v1.1**

Planned Features:

- Automatic Update Checks
  - Signed Package Verification
  - Staged Rollouts
  - Rollback Support
- 

## **Backend Upgrades**

Deployment Method:

Docker Compose Redeployment

---

## **25. Technical Success Criteria**

Sentinel v1.0 is technically complete when:

### **Endpoint Layer**

- Telemetry Collection Functions Correctly
  - SUES Normalization Functions Correctly
  - SQLite Buffering Functions Correctly
  - Heartbeats Function Correctly
- 

### **Processing Layer**

- Redis Streams Function Correctly

- Detection Engine Generates Alerts
  - Sigma Engine Generates Alerts
  - Incident Correlation Functions Correctly
- 

#### **Data Layer**

- PostgreSQL Stores Data Correctly
  - Neo4j Stores Relationships Correctly
  - Backups Function Correctly
- 

#### **Dashboard Layer**

- Alerts Render Correctly
  - Incidents Render Correctly
  - Threat Hunting Functions Correctly
  - Attack Graphs Render Correctly
  - Reports Generate Correctly
- 

#### **Security Layer**

- TLS Functions Correctly
  - Certificate Pinning Functions Correctly
  - JWT Authentication Functions Correctly
  - RBAC Functions Correctly
- 

#### **Operational Targets**

- Performance Targets Achieved
  - Scalability Targets Achieved
  - Recovery Objectives Achieved
- 

**End of TRD v1.0**